Calculatorz

Arithmetic Expression Evaluator in C++

User's Manual

Version 1.2

[Note: The following template is provided for use with the Unified Process for EDUcation. Text enclosed in square brackets and displayed in blue italics (style=InfoBlue) is included to provide guidance to the author and should be deleted before publishing the document. A paragraph entered following this style will automatically be set to normal (style=Body Text).]

[To customize automatic fields (which display a gray background when selected), select File>Properties and replace the Title, Subject and Company fields with the appropriate information for this document. After closing the dialog, automatic fields may be updated throughout the document by selecting Edit>Select All (or Ctrl-A) and pressing F9, or simply click on the field and press F9. This must be done separately for Headers and Footers. Alt-F9 will toggle between displaying the field names and the field contents. See Word help for more information on working with fields.]

Revision History

Date	Version	Description	Author
04/06/2026	1.0	Initial draft	Eian
4/28/2026	1.1	Final Draft	Eian
5/04/2026	1.2	Final Polish	Eian

Table of Contents

Test Case

1. Purpose

This User's Manual describes how to install, configure, and operate Calculatorz , an arithmetic expression evaluator written in C++. The manual covers basic usage, supported operators, precedence rules, error handling, and frequently asked questions. It is intended for end users who want to evaluate arithmetic expressions through the graphical interface as well as developers integrating the engine directly.

2. Introduction

Calculatorz is a C++ arithmetic expression evaluator that accepts standard infix expressions and returns a computed numeric result. Under the hood, the application is composed of three pipeline stages:

- Tokenizer: scans the raw input string character by character and produces a flat list of typed tokens (numbers, operators, and parentheses).
- Parser: converts the infix token list into postfix order, respecting operator precedence and associativity.
- Evaluator: processes the postfix token list using a stack and returns the final double-precision result.

Key features include:

- All four arithmetic operations: addition (+), subtraction (−), multiplication (*), and division (/).
- Correct operator precedence: multiplication and division are evaluated before addition and subtraction.
- Parentheses support to override default precedence.
- Unary minus support for negative number literals (e.g. -3 * 2).
- Floating-point input and output (e.g. 1.5 + 2.5).
- Descriptive error messages for invalid input, mismatched parentheses, and division by zero.
- A graphical front-end with button input and a history panel showing past expressions and results.

To install, download the provided .exe file and run it directly — no additional setup is required.

3. Getting started

3.1 Installation

1. Download the Calculatorz installer (.exe) from the project distribution page.
2. Double-click the .exe file. No additional runtime or dependency installation is needed.
3. The application window will open immediately.

3.2 Entering an Expression

You can enter expressions using the on-screen buttons or by typing directly into the input field. The supported input characters are:

Character(s)	Meaning

0 – 9	Digit characters forming integer or decimal numbers
.	Decimal point (at most one per number)
+	Addition
-	Subtraction (or unary minus when at the start of an expression or after another operator / open parenthesis)
x	Multiplication
÷	Division
^	Exponential
(	Open parenthesis: increases precedence of enclosed sub-expression
)	Close parenthesis: must match an earlier open parenthesis

3.3 Evaluating and Reading the Result

4. Type or click your expression into the input field (e.g. 3 + 4 * 2).
5. Press the = button or the Enter key to evaluate.
6. The result appears in the output display. For the example above, the result is 11 because multiplication is performed before addition.
7. The expression and its result are added to the history panel.

3.4 Clearing and Correcting Input

- Press C (Clear) to erase the entire current expression.
- Press Backspace or the 'Del' button to delete the last character.
- Retype the corrected expression and press = again.

4. Advanced features

4.1 Operator Precedence

Calculatorz follows the standard mathematical order of operations:

- Parenthesized sub-expressions are evaluated first (innermost first).
- Multiplication (*) and division (/) are evaluated before addition (+) and subtraction (−).
- Operators of equal precedence are evaluated left to right.

Examples:

Expression	Result	Reason
2 + 3 * 4	14	3 * 4 = 12 evaluated first, then 2 + 12

(2 + 3) * 4	20	Parentheses force 2 + 3 = 5 first, then 5 * 4
10 / 2 - 1	4	10 / 2 = 5 first (left-to-right), then 5 - 1
8 / (2 * 2)	2	Inner 2 * 2 = 4 first, then 8 / 4
(4^2) + 1	17	Inner 4 squared first, then 16 + 1

4.2 Unary Minus (Negative Numbers)

A minus sign is treated as a unary negation (rather than binary subtraction) when it appears:

- At the very start of the expression (e.g. -5 + 3).
- Immediately after another operator (e.g. 4 * -2).
- Immediately after an open parenthesis (e.g. (-7 + 1)).

In all other positions, the minus sign is treated as binary subtraction.

4.3 Floating-Point Numbers

Both integer and decimal inputs are accepted. The decimal point (.) may appear anywhere within a number but may not appear more than once in the same number. The result is always returned as a double-precision floating-point value.

Expression	Result
1.5 + 2.5	4.0
10.0 / 4	2.5

4.4 History Panel

Every successfully evaluated expression is stored in the history panel. You can scroll through previous entries and click any item to load it back into the input field for re-evaluation or editing.

5. Troubleshooting

Error Message	Cause	Solution
Division by zero	The expression attempts to divide a number by zero (e.g. 5 / 0).	Change the divisor to a non-zero value.
Invalid expression	The token list is malformed, too few operands for an operator, or operators left over with no operands (e.g. 5 + or 3 4).	Check that every operator has two surrounding operands and that no operands are missing.
Mismatched parentheses	An open parenthesis has no matching close, or a close parenthesis has no matching open (e.g. (3 + 4 or 3 + 4)).	Count opening and closing parentheses to ensure they are balanced.
Invalid character: <c>	The expression contains a character	Remove or replace the unsupported

	that is not a digit, operator, decimal point, or parenthesis (e.g. 3 + x).	character.
Invalid negative number	A minus sign used as unary negation is not followed by a digit or decimal point (e.g. a trailing - at end of input).	Ensure a digit or decimal point immediately follows the negation sign.
Invalid number format	A number contains more than one decimal point (e.g. 3.1.4).	Correct the number so it contains at most one decimal point.

6. Examples

Expression	Result	Notes
3 + 4	7	Basic addition
10 - 3	7	Basic subtraction
6 * 7	42	Basic multiplication
15 / 4	3.75	Floating-point division
2 + 3 * 4	14	Precedence: multiply before add
(2 + 3) * 4	20	Parentheses override precedence
-5 + 3	-2	Unary minus at start
4 * -2	-8	Unary minus after operator
(8 + 2) / (3 - 1)	5	Multiple parenthesised groups
1.5 * 4	6.0	Floating-point multiplication

7. Glossary of terms

Term	Definition
Infix notation	The conventional way of writing arithmetic expressions where operators appear between their operands (e.g. 3 + 4).
Postfix notation (RPN)	A notation where operators follow their operands (e.g. 3 4 +). Used internally by the evaluator.
Token	The smallest meaningful unit produced by the tokenizer: a number, an operator, or a parenthesis.
Tokenizer	The component that converts a raw expression string into a list of tokens.
Parser	The component that converts an infix token list to postfix order.
Evaluator	The component that processes a postfix token list with a stack and computes the final numeric result.
Unary minus	A minus sign that negates a single operand rather than subtracting two operands.
Operator precedence	Rules that determine the order in which operators are evaluated (* and / before

	+ and -).
Floating-point	A way of representing real numbers in binary. Calculatorz uses double precision (64-bit) throughout.

8. FAQ

Q: Can I use exponentiation (^) or square root?

A: Not in the current version. Calculatorz supports the four basic arithmetic operators only. Exponentiation and other functions may be added in a future release.

Q: Can I chain many operations in one expression?

A: Yes. There is no hard limit on expression length. Expressions such as 1 + 2 * 3 - 4 / 2 + (5 - 1) are fully supported.

Q: What happens if I press = with an empty input field?

A: An empty expression produces no tokens, so the evaluator returns an "Invalid expression" error. Simply enter a valid expression and try again.

Q: Is there a way to reuse a previous result?

A: Yes — click any entry in the history panel to load it back into the input field. You can then edit it and re-evaluate.

Q: Are spaces allowed in expressions?

A: Yes. The tokenizer skips all whitespace characters, so 3 + 4 and 3+4 are treated identically